

当前项目总架构说明

Jittor 点云降噪项目架构打印版

打印版说明

A4 纵向排版，适合纸质审阅和手写标注。

主线入口、候选链路、官方 VM 补丁和实验脚本已分层说明。

源文件：docs/PROJECT_ARCHITECTURE_CN.md

生成时间：2026-05-20 11:12

内容导航

■ 当前项目总架构说明

- 1. 一句话总览
- 2. 推荐阅读顺序
- 3. 顶层目录作用
- 4. 核心模型代码结构
- 5. 数据、外部产物和忽略规则
- 6. 当前候选和结论口径
- 7. 常见工作流
- 8. 维护建议

当前项目总架构说明

更新时间：2026-05-20

本文面向后来读代码、复现实验、整理提交候选的人，目标是说明当前仓库里“哪些文件负责什么”。它不是比赛结论文档，也不把研究脚本提升为推荐方案。当前仓库仍处于研究收束期，阅读时应优先按“主线入口、候选链路、实验脚本、历史归档”四层理解。

1. 一句话总览

这个项目是 Jittor 点云降噪比赛仓库，核心目标是从 noisy point cloud 生成正式提交格式的 ``denoised.npy``。仓库里同时保留了三类东西：

- 可复现主线：环境、训练、推理、提交检查、候选登记。
- 官方 starter code 及 VM 修补：用于官方 VM baseline、fixed-stitch、streaming stitching。
- 研究实验脚本：PW-SENEL、ST-AAS、router、LIR、GARA-D、fixed075/proxy/blend 等探索。

当前最重要的边界是：

- 主训练/推理代码：``denoise_baseline.py``
- 推荐候选推理入口：``scripts/unified_predict.py``
- 提交包检查入口：``scripts/check_submission.py``
- 候选评估套件：``scripts/evaluate_candidate_suite.py``
- 候选登记入口：``scripts/candidate_registry.py``
- 官方 VM 补丁说明：``docs/OFFICIAL_VM_PATCHES.md``
- 候选事实来源：``experiments/candidate_registry.*``

2. 推荐阅读顺序

第一次接手项目时建议按下面顺序看：

顺序	文件	作用
1	<code>`README.md` / <code>`README_CN.md`</code></code>	项目背景、方法概览、基本入口
2	<code>`docs/REPOSITORY_STRUCTURE.md`</code> / <code>`_CN.md`</code>	仓库边界、哪些文件是复现必需
3	<code>`docs/repo_boundary_audit_20260519.md`</code>	当前主线/实验/归档边界判断
4	<code>`docs/CANDIDATE_REGISTRY.md`</code>	候选登记和当前 best artifact 口径
5	<code>`docs/OFFICIAL_VM_PATCHES.md`</code>	官方 VM 不是纯原版，里面有 documented patches
6	<code>`denoise_baseline.py`</code>	自写降噪模型、训练、预测、打包入口
7	<code>`scripts/README_CN.md`</code>	scripts 目录的工程脚本分组

3. 顶层目录作用

```
.
├── denoise_baseline.py
├── configs/
```

```

├── scripts/
├── starter_code/
├── docs/
├── docs/experiments/
├── experiments/
├── analysis/
├── results/
├── archive/
├── README*.md
├── requirements.txt
└── LICENSE / NOTICE / CITATION.cff

```

3.1 `denoise\_baseline.py`

自写 Jittor 降噪主代码，也是 baseline / ablation 训练和直接预测的核心入口。

主要职责：

- 读取 YAML config/profile。
- 构造训练数据：从 clean OBJ 采样点，合成 noisy/clean 训练对。
- 定义 `ResidualDenoiser` 模型。
- 实现 PW-SENEL、PW-SENEL v2、ST-AAS、move gate、adaptive clip 等开关模块。
- 训练 checkpoint。
- 从 `dataset\_test\_noisy/shapenet/\*/noisy.npy` 预测 `denoised.npy`。
- 将预测目录打包为正式提交 zip。
- 做轻量 zip 路径校验。

它的 CLI 模式：

```

--mode train      训练自写 ResidualDenoiser
--mode predict    加载 checkpoint, 对 test noisy 生成 denoised.npy
--mode zip        把 out_dir 打成提交 zip
--mode validate-zip 轻量检查 zip 内路径结构

```

注意：

- 完整提交校验请用 `scripts/check\_submission.py`。
- 正式候选推荐走 `scripts/unified\_predict.py`，不是直接依赖历史 router 脚本。
- 这个文件会 import Jittor，因此 NumPy-only 工具不应依赖它。

3.2 `configs/`

自写降噪模型的实验配置。它分为两层：

- `configs/denoise\_\*.yaml`：算法/实验配置。
- `configs/profiles/\*.yaml`：机器、显存、训练规模相关覆盖项。

主要配置：

文件	作用
`configs/denoise_baseline.yaml`	最基础 residual denoiser 配置
`configs/denoise_smoke.yaml`	快速 smoke 测试配置
`configs/denoise_pwsenel.yaml`	PW-SENEL v1 ablation

`configs/denoise_pwsenel_v2*.yaml`	PW-SENEL v2、clip、adaptive clip 系列
`configs/denoise_staas_v0.yaml`	ST-AAS v0 几何分支
`configs/denoise_staas_v1_fusion.yaml`	ST-AAS v1 fusion 研究配置
`configs/denoise_staas_v2_gate.yaml`	ST-AAS v2 gate 研究配置
`configs/denoise_staas_v2_highnoise_expert.yaml`	high-noise expert 研究配置
`configs/denoise_move_gate.yaml`	学习式 move gate
`configs/denoise_noise_aware_move_gate*.yaml`	noise-aware move gate 系列
`configs/denoise_hybrid_safe_strong*.yaml`	safe/strong hybrid router 研究配置
`configs/denoise_lir_v1_t2_fast_ablation.yaml`	LIR 快速消融配置

profiles:

文件	作用
`configs/profiles/local_dev.yaml`	本地开发、快速 debug
`configs/profiles/rtx4050.yaml`	RTX 4050 类机器配置
`configs/profiles/rtx5060ti.yaml`	RTX 5060 Ti 类机器配置
`configs/profiles/a6000.yaml`	A6000 正式/大规模训练配置
`configs/profiles/a6000_fast_ablation.yaml`	A6000 快速消融
`configs/profiles/a6000_fast_move_gate.yaml`	A6000 move gate 快速实验

配置合并顺序:

base config -> profile override -> 显式 CLI 参数

也就是说，临时命令行参数应该能覆盖 YAML。

3.3 `scripts/`

工程脚本目录。当前没有把所有研究脚本移动到子目录，主要是为了不破坏已有路径和远端 A6000 工作流。阅读时应按分组理解。

3.3.1 主线工程入口

脚本	作用
`scripts/env.sh`	环境入口。设置 `PROJECT_ROOT`、`PYTHON`、`CC/CXX`、`DISABLE_MULTIPROCESSING`、`JITTER_HOME` 等。Jitter cache 默认写入 `jitter_home/`。
`scripts/install_deps.sh`	安装 `requirements.txt` 依赖。
`scripts/check_env.py`	分层环境检查：Python/NumPy、Jitter CPU、Jitter CUDA。用于避免 NumPy-only 工具被 CUDA 编译问题拖挂。
`scripts/check_data.py`	数据目录和样本结构检查。

`scripts/train.sh`	推荐训练 wrapper，最终调用 `denoise_baseline.py --mode train`。
`scripts/predict.sh`	baseline 预测 wrapper，适合复现 baseline，不是正式候选唯一入口。
`scripts/check_submission.py`	正式提交 zip 校验：路径、文件数、shape、dtype、finite、test_root 对齐。
`scripts/collect_runs.py`	汇总实验 run 信息。

3.3.2 正式候选链路

脚本	作用
`scripts/unified_predict.py`	当前推荐候选推理入口。负责加载 safe/strong checkpoint、noisy-only 路由、LIR/gated-LIR、写 `denoised.npy`、生成 zip、写 routes.csv、可选追加 registry。
`scripts/evaluate_candidate_suite.py`	hidden-test-safe 候选 artifact 体检。检查 zip、hash、size、shape、dtype、位移分布、与基准 zip 的差异。它不声称官方 CD/P2S。
`scripts/candidate_registry.py`	候选登记脚本。写 `experiments/candidates.jsonl`、`candidate_registry.csv`、`candidate_registry.md`。
`scripts/run_official_eval.py`	官方 `starter_code/evaluate.py` 包装器。捕获输出并写 JSON sidecar，供 registry 记录。
`scripts/package_official_vm_outputs.py`	将官方 VM 输出树整理成正式提交 zip 路径：`shapenet/<category>/<model>/denoised.npy`。
`scripts/check_official_chain_dry_run.sh`	不重新推理，只检查已有 official VM fixed-stitch 输出能否打包并通过 submission check。

正式候选推荐流程：

```
source scripts/env.sh
"$PYTHON" scripts/unified_predict.py --name <candidate> --out-dir <dir> --zip <zip>
"$PYTHON" scripts/check_submission.py <zip> --test-root dataset_test_noisy --require-float32
"$PYTHON" scripts/evaluate_candidate_suite.py --candidate <name>==<zip>
"$PYTHON" scripts/candidate_registry.py --name <candidate> --zip <zip> --submission-check passed --conclusion
"<结论>"
```

3.3.3 评估、router、噪声估计

脚本	作用
`scripts/evaluate_cd.py`	在本地 clean OBJ 上合成 noisy，计算 synthetic CD。用于快速 sanity check，不是官方分数。
`scripts/evaluate_noise_estimator.py`	噪声/roughness 估计器评估。
`scripts/evaluate_router.py`	router 结果汇总分析。
`scripts/predict_router.py`	旧 router 推理入口。当前属于历史/诊断路径，新候选优先用 `unified_predict.py`。

`scripts/summarize_router_stress.py`	router stress 输出汇总。
--------------------------------------	---------------------

3.3.4 GARA-D / adapter / proxy 研究脚本

这些脚本用于验证 GARA-D 或 fixed075-base adapter 思路，不是默认正式提交路径。

脚本	作用
`scripts/train_garad_v0.py`	GARA-D v0 synthetic smoke trainer/evaluator。用合成 clean/noisy/base 验证 bounded directional adapter 是否有学习信号。
`scripts/train_garad_fixed075_proxy.py`	fixed075 proxy base 上的 GARA-D 训练/评估实验。
`scripts/train_garad_vm_base_phase1.py`	VM base phase-1 GARA-D 训练实验。
`scripts/garad_identity_adapter.py`	fixed075-base identity adapter。验证 adapter 打包链路零漂移。
`scripts/garad_bounded_tiny_adapter.py`	fixed075-base tiny bounded residual delta artifact smoke。用于风险体检，不是推荐提交。
`scripts/evaluate_bounded_tiny_proxy_gt.py`	bounded tiny proxy 的 GT/本地评估辅助。
`scripts/summarize_garad_adapter_matrix.py`	汇总 GARA-D adapter matrix 结果。

根目录 `run\_garad\_v01\_\*.sh` 是这些研究脚本的批量实验 wrapper:

脚本	作用
`run_garad_v01_base_modes.sh`	比较不同 synthetic base mode
`run_garad_v01_bridge_cd.sh`	bridge-CD synthetic smoke 批量运行
`run_garad_v01_paired_noise.sh`	paired-noise 诊断
`run_garad_v01_paired_offset.sh`	paired-offset 诊断
`run_garad_v01_paired_smooth.sh`	paired-smooth 诊断
`run_garad_v01_healthy_base_confirm.sh`	healthy base confirmation
`run_garad_v01_fixed075_proxy_matrix.sh`	fixed075 proxy matrix
`run_garad_v01_fixed075_proxy_confirm_repeats.sh`	fixed075 proxy repeat confirmation
`run_garad_v01_smoothing_adapter_matrix.sh`	smoothing adapter matrix
`run_garad_v01_smoothing_adapter_confirm_repeats.sh`	smoothing adapter repeat confirmation

这些 root 脚本通常写死某些本机路径或 analysis 输出目录，应视为研究期 workflows。

3.3.5 Blend / P0 / artifact 研究脚本

脚本	作用
`scripts/adaptive_blend_probe.py`	对 stream/VM 与 LIR 两个提交 zip 做 noisy-only adaptive blend，生成候选 zip 和 diagnostics。

`scripts/p0_geometry_blend_gate.py`	P0 几何置信 adaptive blend gate。围绕 fixed075 权重做小范围重分配。
`scripts/scan_fixed075_proxy_base.py`	fixed075 proxy base 扫描。
`scripts/scan_smoothing_base.py`	smoothing base 扫描。

这些脚本的输出应先进入 `analysis/` 和候选 suite，只有有证据后才通过 registry 推广。

3.3.6 几何门控、校准、原型

脚本	作用
`scripts/prototype_move_scale_gate.py`	StraightPCF-style bounded move-scale gate 的 Jitter 算子兼容原型。
`scripts/calibrate_move_scale_refs.py`	move-scale gate 的 roughness / KNN spacing 参考值校准。
`scripts/inspect_official_vm_patch_coverage.py`	官方 VM patch 覆盖诊断；检查 FPS/KNN patch 是否漏点。

3.3.7 A6000 / 远端机器脚本

这些脚本服务 `/workspace/freshman` 类 A6000 环境，默认不是跨平台入口。

脚本	作用
`scripts/activate_a6000_workspace.sh`	激活 A6000 工作区环境。
`scripts/a6000_smoke.sh`	A6000 smoke。
`scripts/a6000_phase1_update_check.sh`	A6000 phase update check。
`scripts/a6000_denoise_baseline_full_predict.sh`	自写 `denoise_baseline.py` A6000 全量预测、打包、校验。
`scripts/a6000_official_vm_full_predict.sh`	官方 VM 全量预测链路。
`scripts/a6000_official_vm_fixed_stitch_predict.sh`	官方 VM fixed-stitch 全量预测、打包、校验。
`scripts/a6000_router_stress_smoke.sh`	router stress smoke。
`scripts/a6000_lir_v1_t2_fast_ablation.sh`	LIR v1 T=2 快速消融。

3.3.8 环境和辅助脚本

脚本	作用
`scripts/gpu_profile.py`	根据 GPU 给 profile 建议。
`scripts/freeze_env.sh`	冻结环境信息，便于复现记录。
`scripts/make_wheelhouse.sh`	准备离线依赖 wheelhouse。
`scripts/check_official_quick_outputs.sh`	检查官方 quick outputs。

3.3.9 Smoke 脚本

`scripts/smoke/` 保存原先根目录下的 smoke wrapper：

脚本	作用
`scripts/smoke/run_config_smoke.sh`	配置 smoke
`scripts/smoke/run_train_baseline.sh`	baseline 训练 smoke
`scripts/smoke/run_denoise_smoke.sh`	denoise smoke
`scripts/smoke/run_denoise_predict_smoke.sh`	denoise predict smoke

这些是快速验证用，不是正式实验入口。

3.4 `starter\_code/`

官方 starter code 目录，但当前不是完全 untouched official code。核心文件 `starter\_code/src/model/vm.py` 有 documented patches，详见 `docs/OFFICIAL\_VM\_PATCHES.md`。

主要结构：

```

starter_code/
├── run.py
├── evaluate.py
├── configs/
├── datalist/
├── datalist_quick/
├── src/data/
├── src/model/
└── src/system/

```

3.4.1 顶层文件

文件	作用
`starter_code/run.py`	官方 starter 训练/预测总入口，按 task config 组装 data/model/system/transform。
`starter_code/evaluate.py`	官方评估入口。`scripts/run_official_eval.py` 会包装它并提取分数。
`starter_code/check_quick_outputs.py`	检查 quick 输出。
`starter_code/run_baseline_quick.sh`	官方 quick baseline wrapper。
`starter_code/smoke_gcc10.sh`	官方/Jittor 编译环境 smoke 辅助。

3.4.2 `starter\_code/configs/`

官方 starter code 配置拆成五组：

子目录	作用
`configs/data/`	train/predict/debug 数据配置。
`configs/model/`	VM 模型配置，主要是 `vm.yamll`。
`configs/system/`	训练/预测系统配置，例如 batch、writer、runner。
`configs/task/`	task 总配置，引用 data/model/system/transform。

`configs/transform/`	数据 transform 配置。
----------------------	------------------

当前 `starter\_code/configs/model/vm.yaml` 是官方 VM 主要配置。历史 distance-gate 相关配置已经归档到 `archive/deprecated\_vm\_distance\_gate\_configs\_20260519/`。

3.4.3 `starter\_code/src/data/`

官方数据层：

文件	作用
`asset.py`	单个样本/资产的数据结构。
`datapath.py`	数据路径解析。
`dataset.py`	数据集构造和读取。
`augment.py`	数据增强。
`transform.py`	transform 逻辑。
`spec.py`	数据模块基类/接口。
`utils.py`	数据工具函数。

3.4.4 `starter\_code/src/model/`

官方模型层：

文件	作用
`feature.py`	官方特征提取模块，`denoise_baseline.py` 也复用其中的 `FeatureExtraction` 和 KNN 工具。
`vm.py`	VelocityModule baseline。当前包含 fixed-stitch coverage repair 和 streaming stitching patch。
`parse.py`	按配置解析模型。
`spec.py`	模型基类/接口。

`vm.py` 当前 patch 边界：

- fixed-stitch coverage repair：修复原 patch stitching 可能漏点的问题。
- streaming fixed-stitch：大点云避免构造 `all\_dists(P,N)`。
- distance/magnitude gate：历史实验分支，当前不作为官方 fixed-stitch 主线。

3.4.5 `starter\_code/src/system/`

官方训练/预测系统层：

文件	作用
`vm.py`	VM 训练/预测 system。
`parse.py`	system 配置解析。
`spec.py`	system 基类/接口。

3.5 `docs/`

文档目录。建议把“结论”和“过程”分开读：

文件	作用
`docs/REPOSITORY_STRUCTURE.md` / `_CN.md`	仓库结构和复现边界。
`docs/repo_boundary_audit_20260519.md`	2026-05-19 的仓库边界审计。
`docs/CANDIDATE_REGISTRY.md`	候选登记机制和当前 best 口径。
`docs/OFFICIAL_VM_PATCHES.md`	官方 VM patch 说明。
`docs/GPU_PROFILES.md`	profile 和硬件环境说明。
`docs/VSCode_A6000.md`	VSCode/A6000 使用说明。
`docs/FORMAL_COMPETITION_EXECUTION_CHECKLIST.md`	历史正式比赛执行 checklist，已偏归档性质。
`docs/ST_AAS_EXECUTION_CHECKLIST.md`	ST-AAS 相关执行 checklist，需结合当前 registry 判断。

3.6 `docs/experiments/`

精选实验报告目录，只保存少量有价值的 markdown 结论，不保存大 zip/csv/cache。

文件	作用
`docs/experiments/README.md`	精选实验报告索引。
`phase0_artifact_audit_20260519.md`	phase0 artifact 审计。
`garad_identity_result_20260519.md`	GARA-D identity adapter 零漂移验证。
`bounded_tiny_proxy_eval_20260519.md`	bounded tiny proxy 风险评估。
`official_vm_fixed_stitch_pressure_20260516.md`	官方 VM fixed-stitch/streaming 压力测试报告。

3.7 `experiments/`

实验元数据、候选登记和部分 checkpoint 输出目录。

最重要的是：

```
experiments/candidates.jsonl
experiments/candidate_registry.csv
experiments/candidate_registry.md
```

它们是候选事实来源。当前 registry 里记录了 fixed075、官方 VM anchor、LIR、noise gate、P0、GARA-D identity/bounded tiny 等 artifact 的状态和结论。

其他 `experiments/denoise\_\*` 目录通常是 checkpoint、训练日志或 run 输出，默认不应当作为公开复现源直接依赖。复现应依赖脚本、配置、registry 和必要的外部 artifact 路径。

3.8 `analysis/`

研究工作区，包含大量临时分析结果、candidate suite 输出、zip 诊断、proxy 评估、扫描结果。

原则：

- 可以保留在本机用于复盘。
- 不应作为主线复现依赖。
- 大 zip、csv、cache、npz、summary 默认不应进入公开主线。
- 重要结论应迁移成 `docs/experiments/\*.md`。

3.9 `results/`

预测输出目录。通常包含 `shapenet/<category>/<model>/denoised.npz` 树。它是运行产物，不是源代码。

常见来源：

- `denoise\_baseline.py --mode predict`
- `scripts/unified\_predict.py`
- 官方 VM wrapper

生成提交 zip 后仍应运行 `scripts/check\_submission.py`。

3.10 `archive/`

归档目录。当前主要有：

```
archive/deprecated_vm_distance_gate_configs_20260519/
```

它保存 deprecated VM distance-gate configs，避免旧实验配置混入当前官方 VM fixed-stitch 主线。

4. 核心模型代码结构

4.1 自写 baseline：`denoise\_baseline.py`

核心类/函数：

名称	作用
`ObjDenoiseDataset`	从 clean OBJ 采样 clean 点云，并合成 noisy 输入。
`BatchPrefetcher`	后台准备 NumPy batch，缓解 OBJ/trimesh 输入瓶颈。
`PWSENEL`	PW-SENEL v1：softmax 邻域筛噪 + maxpool 边缘锁定。
`PWSENELv2Gate`	PW-SENEL v2：显式 noise_conf / edge_conf，对 offset 做门控。
`STAASv0`	结构张量引导的自适应 softmax 几何残差分支。
`ResidualDenoiser`	主模型：FeatureExtraction -> offset head -> optional gates/clip/geometry branch。
`chamfer_l2`	Jittor 版 Chamfer-L2 训练/评估损失。
`train`	训练入口。
`predict_points_in_chunks`	大点云 chunk 推理。
`predict`	测试集预测入口。

`make_zip`	生成提交 zip。
`validate_zip`	轻量 zip 路径检查。
`apply_config`	合并 config/profile/CLI。

模型主路径：

```
noisy xyz
-> FeatureExtraction
-> optional PW-SENEl feature refinement
-> residual offset head
-> optional move_gate / PW-SENElv2 gate / adaptive_clip / residual_clip
-> optional ST-AAS geometry residual
-> pred = noisy + offset
```

4.2 官方 VM: `starter\_code/src/model/vm.py`

核心类/函数：

名称	作用
`VelocityModule`	官方 VM baseline 模型。
`farthest_point_sampling`	FPS seed 选择。
`knn_points`	patch KNN 构造。
`patch_based_denoise`	dense fixed-stitch patch 推理，当前带 coverage repair。
`_streaming_best_assignment`	O(N) streaming 最优 patch 分配。
`patch_based_denoise_streaming`	大点云 streaming fixed-stitch 推理。

官方 VM 的定位：

- 它是 baseline/reference，不是当前所有研究的唯一主线。
- fixed-stitch 是工程修复，保证输入输出点数一一对应。
- streaming 是显存安全补丁，不是分数提升声明。
- 任何 VM patch 都应查 `docs/OFFICIAL\_VM\_PATCHES.md`。

5. 数据、外部产物和忽略规则

这些目录/文件通常不进 git：

路径/类型	原因
`dataset_train/`	官方训练数据，大文件，外部准备。
`dataset_test_noisy/`	官方测试 noisy 数据，大文件，外部准备。
`analysis/`	研究工作区，包含大量临时产物。
`results/`	预测输出。
`*.zip`	提交包或中间 artifact。

`*.pkl`	checkpoint。
`*.npy` / ``*.npz`	点云或缓存。
`.venv/` / `starter_code/.venv/`	虚拟环境。
`.jittor_home/`	Jittor cache。

复现不应依赖某个本机 `analysis/` 目录，而应依赖：

```
脚本 + config/profile + checkpoint/artifact 路径 + candidate registry + curated docs
```

6. 当前候选和结论口径

以 `docs/repo\_boundary\_audit\_20260519.md` 和 `experiments/candidate\_registry.\*` 为准：

类别	口径
当前 best official artifact	fixed075 / `blend_best075_lir025_20260517`，registry 记录 official score `53.32`
官方 VM anchor	fixed-stitch / streaming VM，可作为稳定 baseline/reference
GARA-D identity	adapter 链路验证，零漂移 smoke，不是得分提升
bounded tiny	实验态 risk smoke，不是官方提交推荐
noise gate / P0 plane	有官方/风险证据但未超过 fixed075，不推荐提升为当前主线
旧 router / raw LIR	历史或诊断路径

任何“推荐提交”判断都应指向 registry，而不是单个脚本名或旧 README 示例。

7. 常见 workflow

7.1 环境检查

```
source scripts/env.sh
"$PYTHON" scripts/check_env.py --level python
"$PYTHON" scripts/check_env.py --level jittor-cpu
"$PYTHON" scripts/check_env.py --level jittor-cuda
```

7.2 训练 baseline 或 ablation

```
source scripts/env.sh
bash scripts/train.sh configs/denoise_baseline.yaml configs/profiles/local_dev.yaml
```

7.3 直接 baseline 预测和打包

```
source scripts/env.sh
"$PYTHON" denoise_baseline.py --config configs/denoise_baseline.yaml --mode predict
"$PYTHON" denoise_baseline.py --config configs/denoise_baseline.yaml --mode zip
"$PYTHON" scripts/check_submission.py result_denoise_baseline.zip --test-root dataset_test_noisy
```

```
--require-float32
```

7.4 正式候选生成和登记

```
source scripts/env.sh
"$PYTHON" scripts/unified_predict.py \
--name <candidate> \
--out-dir results/<candidate> \
--zip result_<candidate>.zip

"$PYTHON" scripts/check_submission.py result_<candidate>.zip \
--test-root dataset_test_noisy \
--require-float32

"$PYTHON" scripts/evaluate_candidate_suite.py \
--candidate <candidate>=result_<candidate>.zip

"$PYTHON" scripts/candidate_registry.py \
--name <candidate> \
--zip result_<candidate>.zip \
--submission-check passed \
--conclusion "<evidence-backed conclusion>"
```

7.5 官方 VM fixed-stitch 链路

```
source scripts/env.sh
bash scripts/a6000_official_vm_fixed_stitch_predict.sh
```

或只检查已有输出树：

```
bash scripts/check_official_chain_dry_run.sh starter_code/tmp_predict_fixed_stitch/tmp/result_official_vm_fixed_stitch_dryrun.zip
```

8. 维护建议

当前仓库应继续遵守这些边界：

- 不大规模删除/移动研究脚本，除非先拆 commit 并确认路径引用。
- 主线文档只推荐明确入口，不把历史实验脚本写成推荐算法。
- `analysis/` 保持工作区属性，重要结论迁移到 `docs/experiments/`。
- `experiments/candidate\_registry.\*` 是候选事实来源，应谨慎维护。
- `starter\_code/src/model/vm.py` 的补丁必须在 `docs/OFFICIAL\_VM\_PATCHES.md` 里说明。
- A6000 脚本可保留本机/远端路径，但要标明 machine-specific。
- 新增实验脚本时，在文件头或 `scripts/README\_CN.md` 中标明是 mainline、candidate、experimental 还是 archive。